

Struktura podataka i algoritmi

VEŽBE 1

TEME

- **Imperativni** i **deklarativni** programski jezici
- Objektno-orijentisani jezici
- Mašinski, asemblerski i viši programski jezici
- Organizacija **modularnih programa**
- Neki jednostavni ali važni **algoritmi**

TIPOVI ZNANJA (programski jezici)

- **Deklarativno znanje** predstavlja **iskaze činjenica**
- **Imperativno znanje** je **uputstvo** kako rešiti (how-to) problem
- **Deklarativni jezici** opisuju rešenje problema na formalan način bez navođenja preciznog programskog toka ili opisuju šta treba izračunati
- **Imperativni jezici** opisuju logiku za rešavanje problema navođenjem sekvence naredbi u programu (programski tok) tj. opisuju **KAKO** nešto rešiti
- **Programiranje** predstavlja „pisanje uputstva“ kako bi se generisale činjenice o nekom problemu

U imperativnim jezicima se rešenje opisuje “ŠTA” koracima, dok se u funkcionalnim/deklarativnim jezicima opisuje “KAKO” koracima

IMPERATIVNI/DEKLARATIVNI jezici

- Program prilikom izvršavanja, prolazi kroz određena stanja koja su definisana promenljivim u memoriji
- Programer u **imperativnom** jeziku određuje način na koji pojedine promenljive menjaju svoja stanja, dok se kod **deklarativnih** jezika ovim pitanjem bave prevodilac ili interpreter na osnovu opisa željenog rezultata
- Većina programskih jezika ima imperativnu prirodu
- Poznatiji deklarativni jezici su **Prolog** i **SQL**
- **Prolog** je jezik opšte namene koji se najčešće koristi u oblasti veštačke inteligencije, dok se jezik **SQL** koristi za formiranje upita putem kojih se manipuliše bazom podataka

IMPERATIVNI/DEKLARATIVNI jezici

- Imperativni jezici razlikuju se po načinu organizacije izvornog koda
- Dva glavna pristupa su:
 - **proceduralni**
 - **objektno orijentisani**
- **Proceduralni jezici** organizuju program po grupama naredbi koje obavljaju određene logičke funkcionalnosti višeg nivoa – procedure (iscrtavanje trougla, rešavanje sistema jednačina itd). Procedura može realizovati svoj zadatak pozivom drugih procedura
- Proceduralni pristup omogućava formiranje biblioteke procedura za rešavanje određene klase problema, a koja se može ponovo upotrebiti u različitim programima kada su potrebne tražene funkcionalnosti
- Najpoznatiji proceduralni jezici su **Basic**, **Cobol**, **Fortran**, **Pascal** i **C**

IMPERATIVNI/DEKLARATIVNI jezici

- **Najveći nedostatak proceduralnog pristupa** je u tome što su procedure, kojima se apstrahuju funkcionalnosti, odvojene od podataka
- Iako svaka procedura ima svoje lokalne promenljive koje samo ona vidi i menja, ključni podaci u programu čuvaju se u globalnim promenljivim koje su svima dostupne pa se mogu neželjeno promeniti iz svake procedure
- Osim sigurnosnog problema, zbog odvojenosti logike obrade od podataka, proceduralni jezici neadekvatno modeluju realne sisteme
- Oni su pogodni za modelovanje pojedinih procesa koji se odvijaju unutar sistema, ali ne i za kompleksne sisteme u celini

Objektno-orijentisani jezici

- **Objektno orijentisani jezici** nastali su iz potrebe da se prevaziđu problemi u modelovanju složenih sistema koji nastaju u proceduralnom pristupu
- Bilo je potrebno razviti pristup koji će povećati produktivnost u pisanju složenih programskih sistema (ponovna upotrebljivost već napisanog koda, jednostavno proširivanje novim funkcionalnostima, olakšano otklanjanje grešaka...)
- U objektno orijentisanom pristupu, program se gradi od **objekata** koji imaju svoje osobine i ponašanje
- Osobine objekta (**atributi**, **svojstva**) realizuju se kao promenljive koje ukazuju na podatke različitih tipova, a ponašanja kroz procedure (**metode**) koje definišu šta objekat može da radi, odnosno šta može da se radi nad njim

Objektno-orijentisani jezici

- Objektni pristup uvodi pojam **klase**
- **Klasa** predstavlja skup objekata sa istim osobinama i istim ponašanjem tj. klasa predstavlja plan po kome se u programu kreiraju objekti.
- Primer koji opisuje sve studente - **klasa Student**: Ova klasa definiše **atribute** (ime, prezime, broj_indeksa...), kao i **metode** (predstavi_se, prijavi_ispit, polaži_ispit...). Mogu se kreirati **objekti** (studenti) koji predstavljaju stvarne studente (npr. Petar_Petrović ili Milica_Milić), a koji imaju iste atribute (sa različitim vrednostima) i isto ponašanje (metode)
- Program se izvršava tako što se kreiraju potrebni objekti koji u interakciji, kroz međusobno pozivanje metoda, realizuju potrebne zadatke
- OO koncept se prirodno uklapa u okvir modelovanja realnog sistema sačinjenog od velikog broja podsistema
- **Najpoznatiji OO jezici su C++, C#, Java, Python, PHP**

Funkcionalno programiranje

- **Funkcionalni pristup programiranju** je po svojim osobinama bliži deklarativnom konceptu
- U funkcionalnom pristupu program se gradi od funkcija koje su slične procedurama (metodama) i kod ovog pristupa nema globalnih promenljivih koje čuvaju stanje programa, već se rezultat rada funkcije prosleđuje sledećoj funkciji na dalju obradu
- Programi pisani na ovaj način često su kraći, a zbog odsustva promenljivih lakše je dokazati njihovu korektnost, ali fon Nojmanov model računara više pogoduje nefunkcionalnom programiranju
- Predstavnik ove grupe programskih jezika je **Haskell**
- **Savremeni jezici primenjuju različite principe koji omogućavaju pisanje i organizaciju programa na različite načine (u Pythonu je moguće programirati samo proceduralno, samo objektno-orijentisano ili koristiti kombinaciju oba pristupa, a podržava i elemente funkcionalnog programiranja koji se mogu mešati sa imperativnim pristupima**

MAŠINSKI, ASEMBLERSKI I VIŠI PROGRAMSKI JEZICI

- Algoritmi se u računarima realizuju odgovarajućim **programom**
- Programi se realizuju korišćenjem različitih sistema simbola (jezika), kojima se izvode pojedine operacije podržane od strane računarskog sistema
- Razmatramo **mašinski**, **asemblerški** i **više programske jezike**
- Procesor računara izvršava program instrukciju po instrukciju, pri čemu se pod instrukcijom podrazumeva jedna od konačno mnogo **elementarnih operacija** podržanih od strane procesora
- Elementarne operacije sastoje se od nizova 0 i 1 koje su smeštene u radnoj memoriji računara (u toku izvršavanja) ili na spoljnoj, trajnoj memoriji (kada računar ne izvršava program ili je ugašen)

MAŠINSKI JEZIK

- Skup svih elementarnih operacija, sa odgovarajućom binarnom formom i pravilima za pristup memorijskim lokacijama, definiše **mašinski jezik za datu familiju procesora**
- Mašinski jezik se sastoji od nizova binarnih reči (0 ili 1) koje predstavljaju instrukcije logičkim sklopovima i podatke koje treba obraditi
- Program napisan u mašinskom jeziku nazivamo izvršni program (izvršni kôd) jer ga računar može neposredno izvršiti
- Mašinski jezik je određen arhitekturom računara, a izvršni program je mašinski zavisn prvenstveno od arhitekture procesora (nije isti za Intel ili AMD procesore)

```
00000000 0000 0001 0001 1010 0010 0001 0004 0128
00000010 0000 0016 0000 0028 0000 0010 0000 0020
00000020 0000 0001 0004 0000 0000 0000 0000 0000
00000030 0000 0000 0000 0010 0000 0000 0000 0204
00000040 0004 8384 0084 c7c8 00c8 4748 0048 e8e9
00000050 00e9 6a69 0069 a8a9 00a9 2828 0028 fdfc
00000060 00fc 1819 0019 9898 0098 d9d8 00d8 5857
00000070 0057 7b7a 007a bab9 00b9 3a3c 003c 8888
00000080 8888 8888 8888 8888 288e be88 8888 8888
00000090 3b83 5788 8888 8888 7667 778e 8828 8888
000000a0 d61f 7abd 8818 8888 467c 585f 8814 8188
000000b0 8b06 e8f7 88aa 8388 8b3b 88f3 88bd e988
000000c0 8a18 880c e841 c988 b328 6871 688e 958b
000000d0 a948 5862 5884 7e81 3788 1ab4 5a84 3eec
000000e0 3d86 dcb8 5cbb 8888 8888 8888 8888 8888
000000f0 8888 8888 8888 8888 8888 8888 8888 0000
00001000 0000 0000 0000 0000 0000 0000 0000 0000
*:
00001030 0000 0000 0000 0000 0000 0000 0000
0000103e
```

MAŠINSKI JEZIK

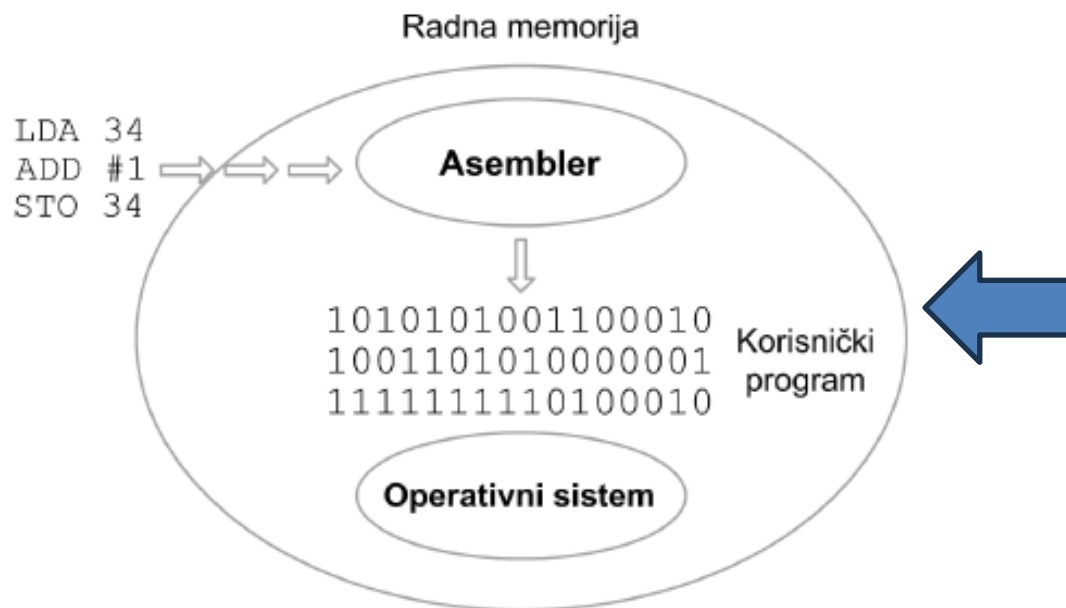
```
10100001 10111100 10010011 00000100
00001000 00000011 00000101 11000000
10010011 00000100 00001000 10100011
11000000 10010100 00000100 00001000
```

It means: $z = x + y;$

- Pisanje programa, kao i njegovo testiranje zahteva detaljno poznavanje arhitekture računara i procesora
- Pisanje programa u mašinskom jeziku predstavlja izuzetno neproduktivan i greškama podložan zadatak
- Korišćenjem elementarnih mašinskih operacija, složeniji zadaci se rešavaju mnogo teže nego kada bi se koristili kompleksniji alati
- Prednosti mašinskog jezika:
 - Najbrže se izvršavaju jer su u binarnoj formi
 - Ne moraju da se prevode jer već postoje u binarnoj formi
 - CPU direktno izvršava mašinski jezik
 - Koristi se u razvoju najbržih računarskih sistema
 - Evolucija računarskih sistema i OS nastala je zbog mašinskog jezika
- Nedostaci mašinskog jezika:
 - Kompleksan za memorisanje i razumevanje
 - Pisanje koda jako vremenski zahtevno i neproduktivno
 - Jako teško otkrivanje grešaka u kodu i programima
 - Veliki broj grešaka tokom pisanja koda
 - Zavisnost od CPU arhitekture

ASSEMBLER

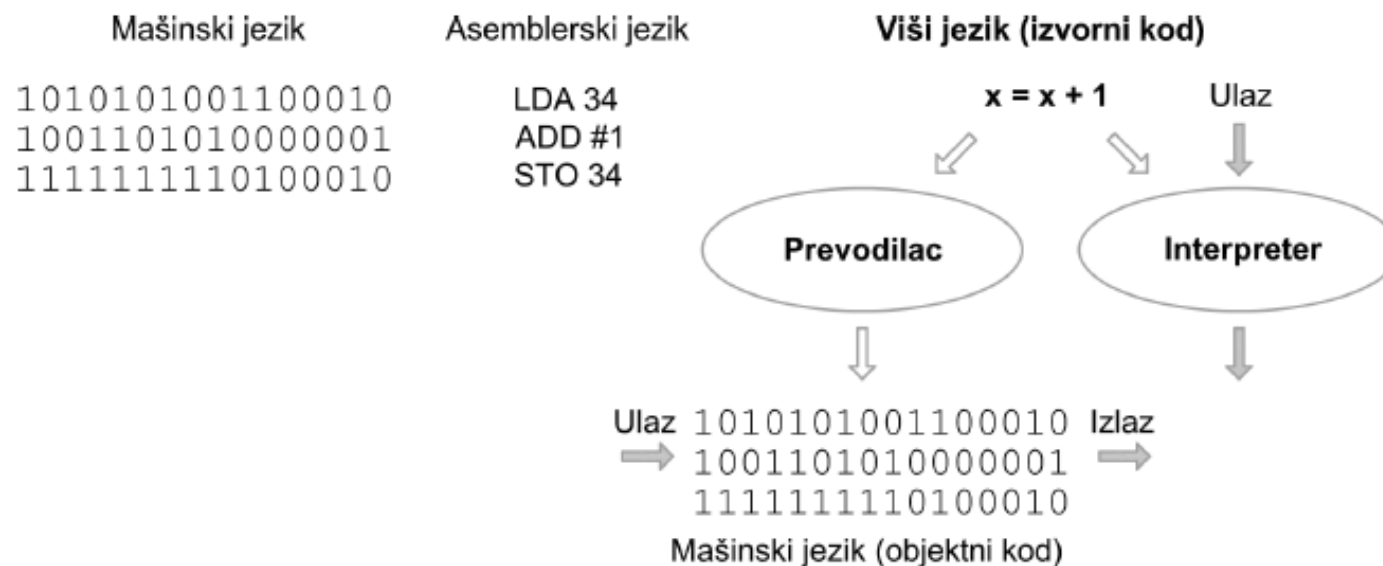
- Zbog složenosti mašinskog koda, osmišljen je **asembler** koji je olakšao proces programiranja upotrebom skraćenica i konstanti koje su se odnosile na pojedine instrukcije, podatke i adrese
- Asemblerski jezik se automatski prevodi u odgovarajuće binarno-kodirane instrukcije; Asemblerski jezik iziskuje postojanje **asemblera**, programa u radnoj memoriji koji prevodi skraćenice jezika u mašinski kod



Asemblerski jezik kao viši nivo apstrakcije u odnosu na mašinski jezik. Prikazan je program koji uvećava sadržaj memorijske lokacije sa adresom 34 za 1, prvo se u akumulator procesora učitava vrednost sa željene lokacije (LDA adresa) a potom se na nju dodaje broj 1 (ADD #1) pa se rezultat upisuje na isto mesto u memoriji (STO 34). Asembler prevodi skraćenice i dekadne (ili heksadekadne) brojne konstante u odgovarajuće mašinske naredbe

VIŠI PROGRAMSKI JEZICI

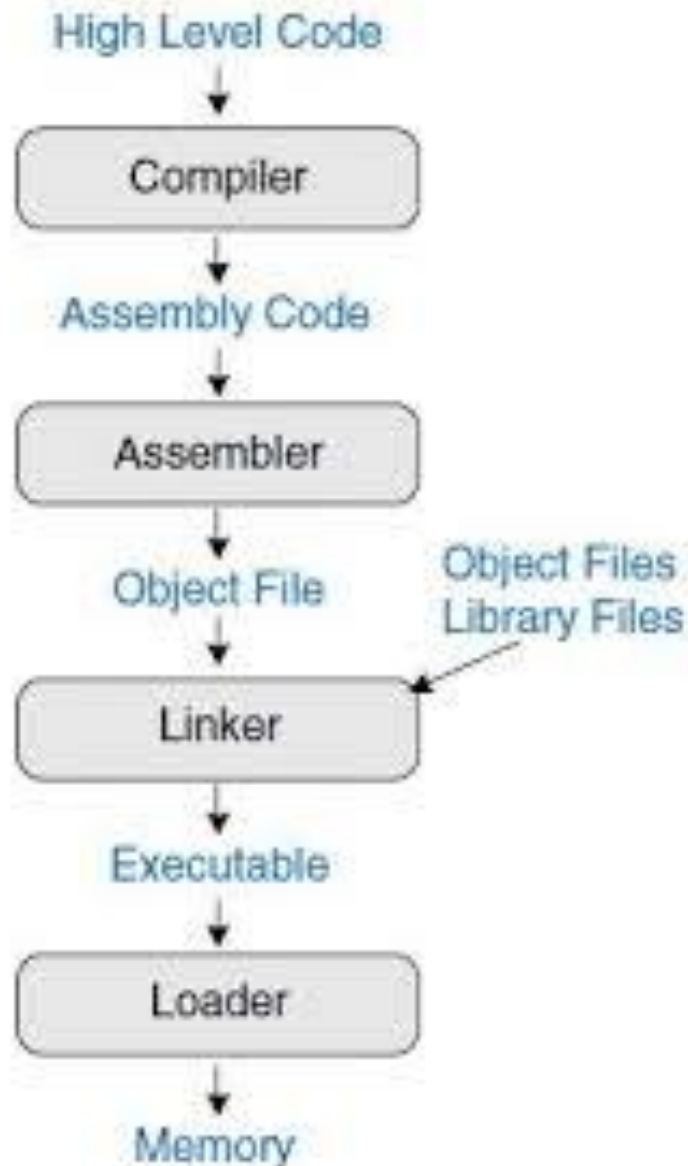
- Asembleri imaju niz nedostataka: potrebno je vršiti detaljizaciju algoritma, zavise od računarske platforme (program napisan na asembleru za jednu platformu ne može bez veće ili manje prerade da se izvršava na drugoj...)
- U želji da se poveća izražajnost, kao i da se omogući što veća nezavisnost od tipa i organizacije sistema, uvodi se novi nivo apstrakcije - **viši programski jezik**
- Viši programski jezici (C, C++, Java, Python, PHP, C# i drugi) uvode čitljive opise za složenije operacije i koncepte, približavajući se prirodnom jeziku
- Viši programski jezik uvodi jezičke konstrukcije koje su bliske matematičkom i prirodnom jeziku, čime se postiže nezavisnost od mašinskog koda konkretne mašine



VIŠI PROGRAMSKI JEZICI

- Program napisan u višem programskom jeziku označava se kao **izvorni kod**
- Izvorni kod je lako razumljiv za čoveka, ali se mora prevesti u mašinski jezik uz pomoć drugog programa – **prevodioca**
- U procesu prevođenja, naredbe višeg programskog jezika zamenjuju se brojnim mašinskim instrukcijama, a dobijeni rezultat je **objektni kod**
- Prenosivost se u praksi ostvaruje kroz postojanje različitih prevodilaca koji isti jezik prevode u mašinske jezike ciljnih računarskih sistema
- Drugi pristup je korišćenje programa - **interpretera**
- Interpreter čita naredbe višeg jezika i izvršava ih jednu po jednu, koristeći sopstveni skup operacija.
- Umesto da se, kao kod prevođenja, program izvršava direktno u procesoru, izvršavaju se direktno instrukcije interpretera koje sprovode željene akcije, što je zbog dodatnog nivoa apstrakcije sporije nego u slučaju prevedenog koda
- Pristup sa interpreterom omogućava da se program brže menja i modifikuje jer u fazi razvoja postoji česta potreba za testiranjem, kako pojedinih delova, tako i celine

Prevođenje programskog kôda



Kompajler je program koji kao ulaz prihvata program napisan u višem programskom jeziku (izvorni kod), a kao izlaz generiše ekvivalentan program u mašinskom kôdu (objektni kod), on je u formi mašinskog kôda koji se može direktno izvršavati na hardveru računara; Objektni program koji se generiše na izlazu kompajlera upisuje se u sekundarnu memoriju

Linker je program za povezivanje, tj. povezuje programske module u jednu celinu

Korisnički program poznat kao **punilac (loader)** smešta program iz sekundarne meorije u glavnu čime je objektni program spreman za izvršenje

Interpreter uzima jednu instrukciju iz programa, analizira je i uslovljava da se sa istim efektom izvrši niz instrukcija nižeg nivoa. Ovaj proces se nastavlja sve dok se ne izvrši kompletan program

ALGORITAM

- 1) Sekvenca jednostavnih **KORAKA**
- 2) Proces **kontrole toka** koji određuje svaki korak kako će biti izvršen
- 3) Potrebno odrediti **uslov zaustavljanja**

NUMERIČKI PRIMER

- Kvadratni koren broja x je y tako da je $y * y = x$
- Početak sa **pretpostavkom**, g
 - 1) Ako je $g * g$ **dovoljno blizu** x , završiti i g je rešenje
 - 2) U suprotnom izvršiti **novu pretpostavku** traženjem sredine za $g+x/g$
 - 3) **Ponoviti** proces dok se ne dođe dovoljno blizu
- Primer: $x = 16$ i početna pretpostavka je $g=3$

g	$g * g$	x / g	$(g + x / g) / 2$
3	9	$16 / 3$	4.17

NUMERIČKI PRIMER

- Kvadratni koren broja x je y tako da je $y^*y = x$
- Početak sa **pretpostavkom**, g
 - 1) Ako je g^*g **dovoljno blizu** x , stop i g je rešenje
 - 2) U suprotnom izvršiti **novu pretpostavku** traženjem sredine za $g+x/g$
 - 3) **Ponoviti** proces dok se ne dođe dovoljno blizu
- Primer: $x = 16$ i početna pretpostavka je $g=3$

g	g^*g	x/g	$(g+x/g) / 2$
3	9	16/3	4.17
4.17	17.36	3.837	4.0035

NUMERIČKI PRIMER

- Kvadratni koren broja x je y tako da je $y^*y = x$
- Početak sa **pretpostavkom**, g
 - 1) Ako je g^*g **dovoljno blizu** x , stop i g je rešenje
 - 2) U suprotnom izvršiti **novu pretpostavku** traženjem sredine za $g+x/g$
 - 3) **Ponoviti** proces dok se ne dođe dovoljno blizu
- Primer: $x = 16$ i početna pretpostavka je $g=3$

g	g^*g	x/g	$(g+x/g) / 2$
3	9	16/3	4.17
4.17	17.36	3.837	4.0035
4.0035	16.0277	3.997	4.000002

RAČUNARI su MAŠINE koje izvršavaju instrukcije prema ALGORITMIMA

- Stvari koje računar radi:
 - Izvršava jednostavne **operacije**
x100 milijardi/sec!
 - **Pamti** rezultate na
x100 GB storage prostora!
- Koji tip kalkulacija?
 - **Ugrađene kalkulacije** u računar, npr. +, -, *, /
 - One koje definiše **sam programer**

RAČUNAR ĆE URADITI SAMO ONO ŠTO
MU SE „KAŽE“ DA URADI

A COMPUTER WILL ONLY DO
WHAT YOU TELL IT TO DO



RAČUNARI su MAŠINE koje IZVRŠAVAJU ALGORITME

- Računar sa **fiksnim programom**
 - Nepromenljivi set algoritama
 - Postojao do 1940
- **Stored program** computer
 - Mašina čuva i izvršava instrukcije
- **Ključna činjenica:** Programi su isto što i ostali tipovi podataka

RAČUNARI SA ČUVANJEM PROGRAMA

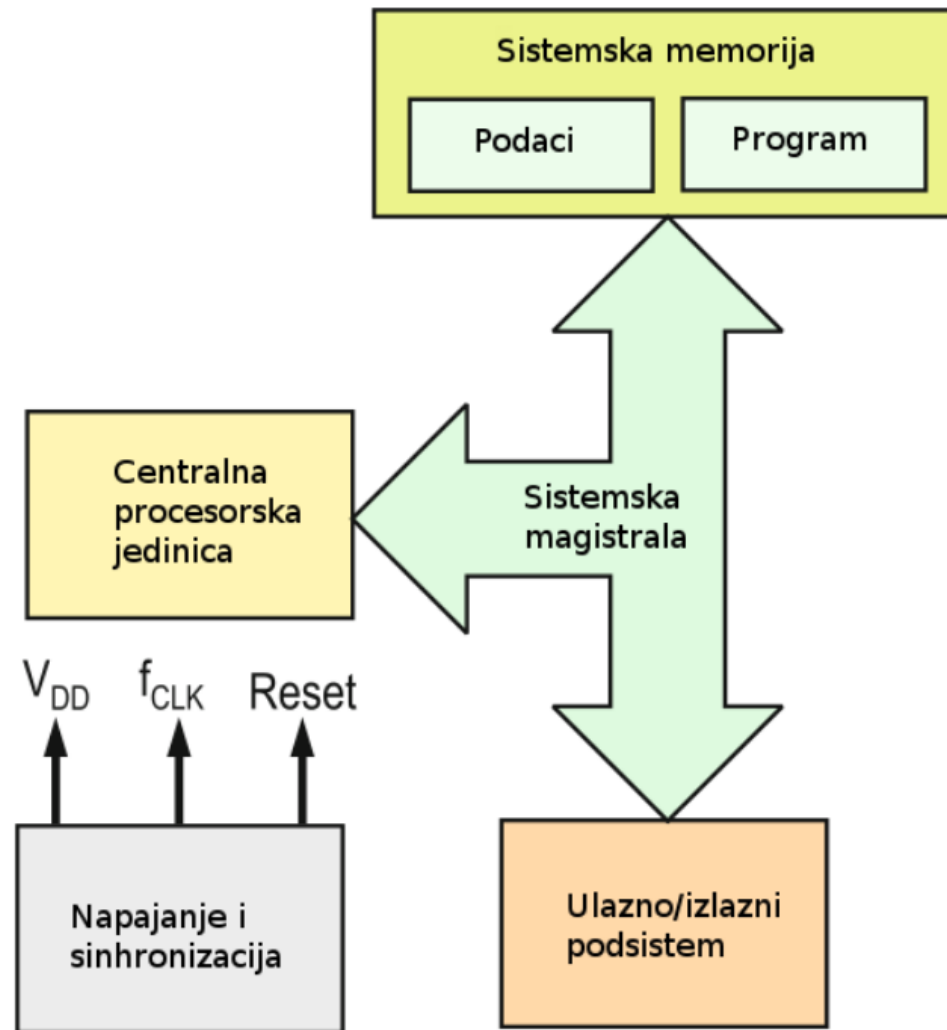
- Sekvenca **instrukcija koja se čuva** u računaru
 - Formira se kao predefinisani set osnovnih instrukcija
 - 1) Aritmetičke i logičke
 - 2) Jednostavni testovi
 - 3) Prenos podataka na različite lokacije računara
- Poseban program (interpreter) **izvršava svaku instrukciju redom kako su programski napsiane**
 - Koristi testove za promenu kontrole toka podataka preko sekvenci
 - Zaustavlja se kada više nema instrukcija ili izvršava instrukciju
ZAUSTAVLJANJA

Arhitektura računara

◆ Računarski sistem zasnovan na **Fon Nojmanovoj arhitekturi** (programi i podaci su smešteni u isti memorijski prostor) sastoji se od sledećih komponenti:

- **Centralna procesorska jedinica** (**CPU**, **procesor**, **mikroprocesor**): Predstavlja ključni deo svakog računarskog sistema, čita instrukcije iz programske memorije, dekoduje ih i izvršava. CPU je zadužen za adekvatno korišćenje perifernih jedinica koje sačinjavaju ulazno/izlazni podsistem (I/O) računara
- **Sistemska memorija** (**RAM memorija**) zadužena je za čuvanje programa i podataka koji se koriste od strane procesora. Računar sadrži dva tipa memorije: programsku memoriju i memoriju podataka
 - ◆ U programsku memoriju smešta se program u formi sekvence mašinskih instrukcija koju procesor izvršava. Program određuje funkcionalnost celokupnog sistema.
 - ◆ Memorija za podatke ima ulogu čuvanja podataka (promenljivih, varijabli) koji se obrađuju tokom izvršavanja programa.

Arhitektura računara



Blok-šema računarskog sistema zasnovanog na Fon Nojmanovoj arhitekturi

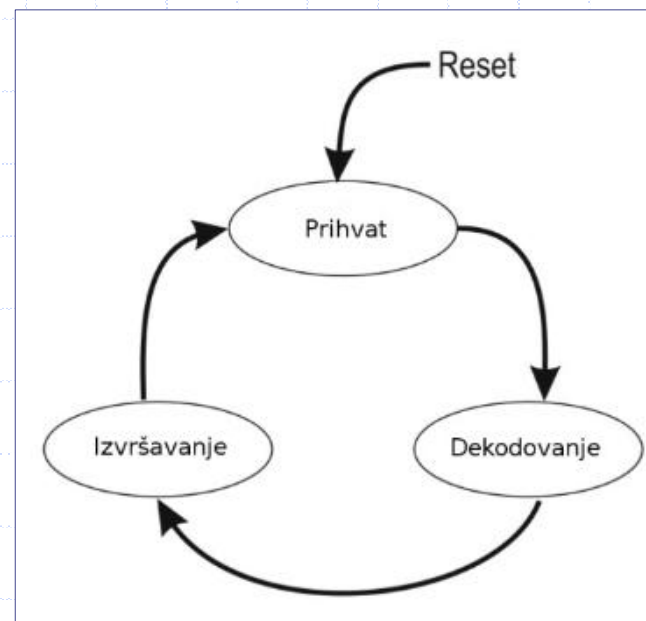
CPU ciklus izvršavanja instrukcija

Kontrolna jedinica (CU) upravlja funkcionalnošću procesora implementirajući konačni automat FSM (*Finite State Machine*) koji ciklično ponavlja tri stanja:

1) **Prihvat instrukcije**: Tokom stanja prihvata nove instrukcije, instrukcija se iz programske memorije učitava u CPU. Programski brojač (PC) obezbeđuje adresu u memoriji sa koje se instrukcija čita. Adresirana instrukcija se preko magistrale podataka prebacuje u registar instrukcija (IR)

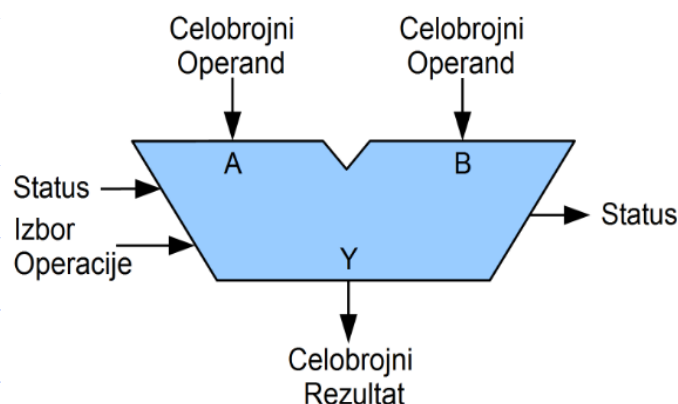
2) **Dekodiranje instrukcije**: Nakon prihvata instrukcije, CU prelazi u stanje dekodiranja, u kome se značenje instrukcije „dešifruje“, a dekodirana informacija se koristi kako bi se slali odgovarajući upravljački signali ka CPU komponentama u cilju izvršavanja aktivnosti predviđenih samom instrukcijom

3) **Izvršavanje instrukcije**: U ovom stanju, CU šalje komande odgovarajućim funkcionalnim jedinicama u okviru CPU u cilju izvršavanja aktivnosti određenih instrukcijom. Na kraju ove faze, sadržaj programskog brojača se inkrementira, kako bi pokazivao na adresu sledeće instrukcije u programskoj memoriji, koja će se izvršiti tokom narednog instrukcijskog ciklusa



CPU ciklus: Fetch-Decode-Execute (FDE)

CPU – ALU (aritmetičko-logička jedinica)



Aritmetičke operacije:

- Sabiranje (sa ili bez prenosa)
- Oduzimanje (sa ili bez pozajmice)
- Inkrement
- Dekrement
- Drugi komplement
- Množenje
- Deljenje

Logičke operacije:

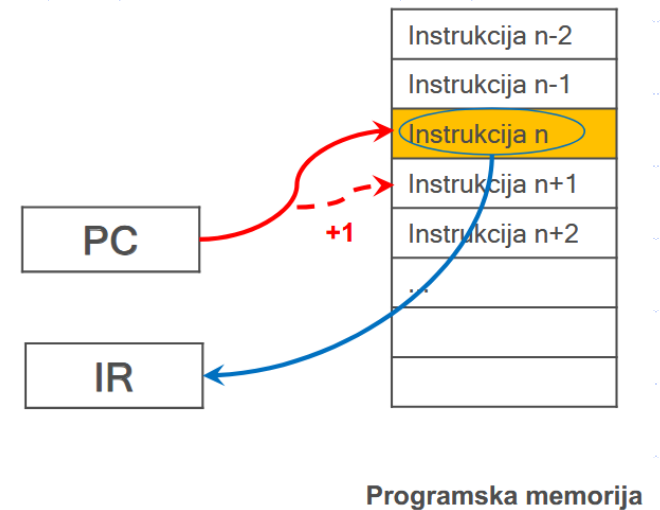
- Bitsko I (AND)
- Bitsko ILI (OR)
- Bitsko EKS-ILI (XOR)
- Bitska negacija (NOT)

Operacije pomeranja:

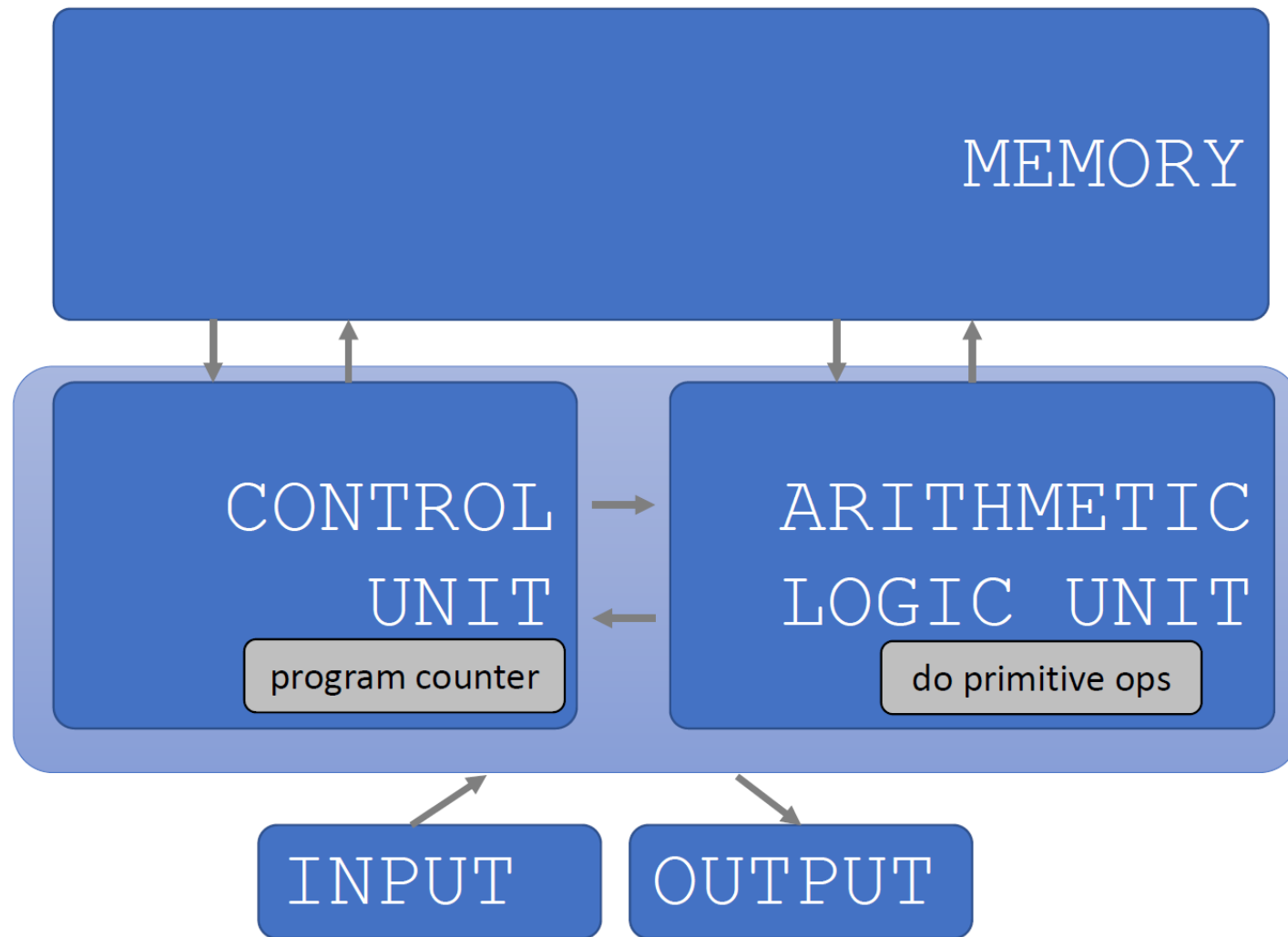
- Pomeranje (ulevo ili udesno)
- Rotacija (ulevo ili udesno)

CPU registri

- Registar instrukcija (**IR**) prima instrukcije iz programske memorije, kako bi CU jedinica mogla da ih dekodira i izvrši. Akcija prenosa instrukcije iz memorije u IR naziva se prihvatanje instrukcije.
- Programski brojač (**PC**) sadrži adresu instrukcije koja se učitava iz programske memorije u instrukcijski registar. Svaki put kada se instrukcija prihvata i dekoduje, automatski se inkrementira vrednost PC registra, kako bi pokazivao na narednu instrukciju koja će se izvršavati tokom sledećeg ciklusa. Izuzetak predstavljaju instrukcije skoka i poziva potprograma.
- Struktura **steka** realizuje se u okviru memorije za podatke, a pokazivač steka (SP) sadrži adresu na kojoj se nalazi podatak koji se trenutno nalazi na vrhu steka (stek se popunjava od nižih ka višim adresama ili obrnuto, a prilikom svakog dodavanja novog podatka na stek, ili preuzimanja podatka sa steka, SP se automatski inkrementira ili dekrementira po **LIFO** principu)
- **Statusni registar** sadrži skup indikatorskih bita (Flags), koji se automatski setuju/resetuju u zavisnosti od rezultata aritmetičke ili logičke instrukcije koju obavlja ALU

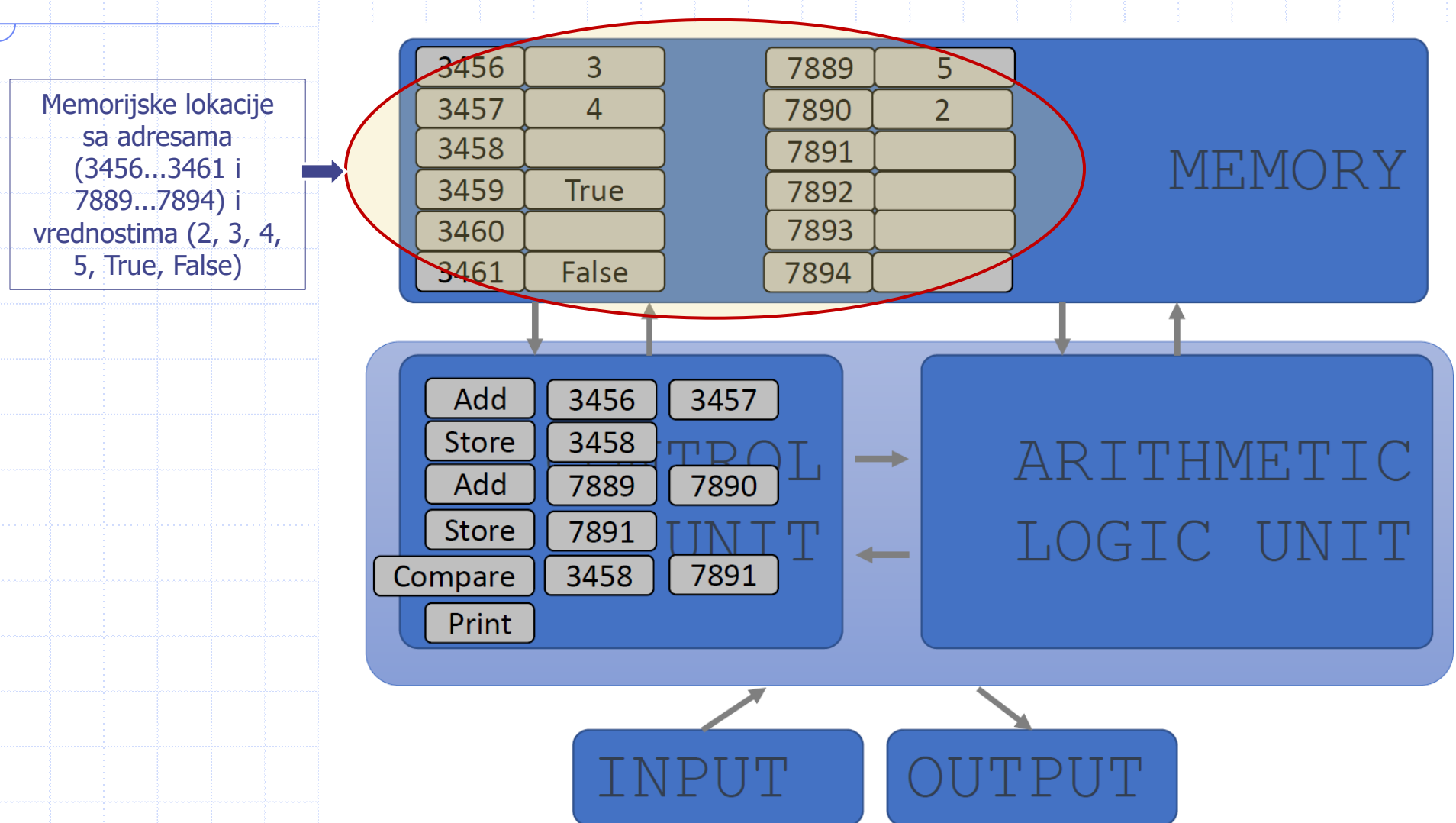


Izvršavanje programa - primer



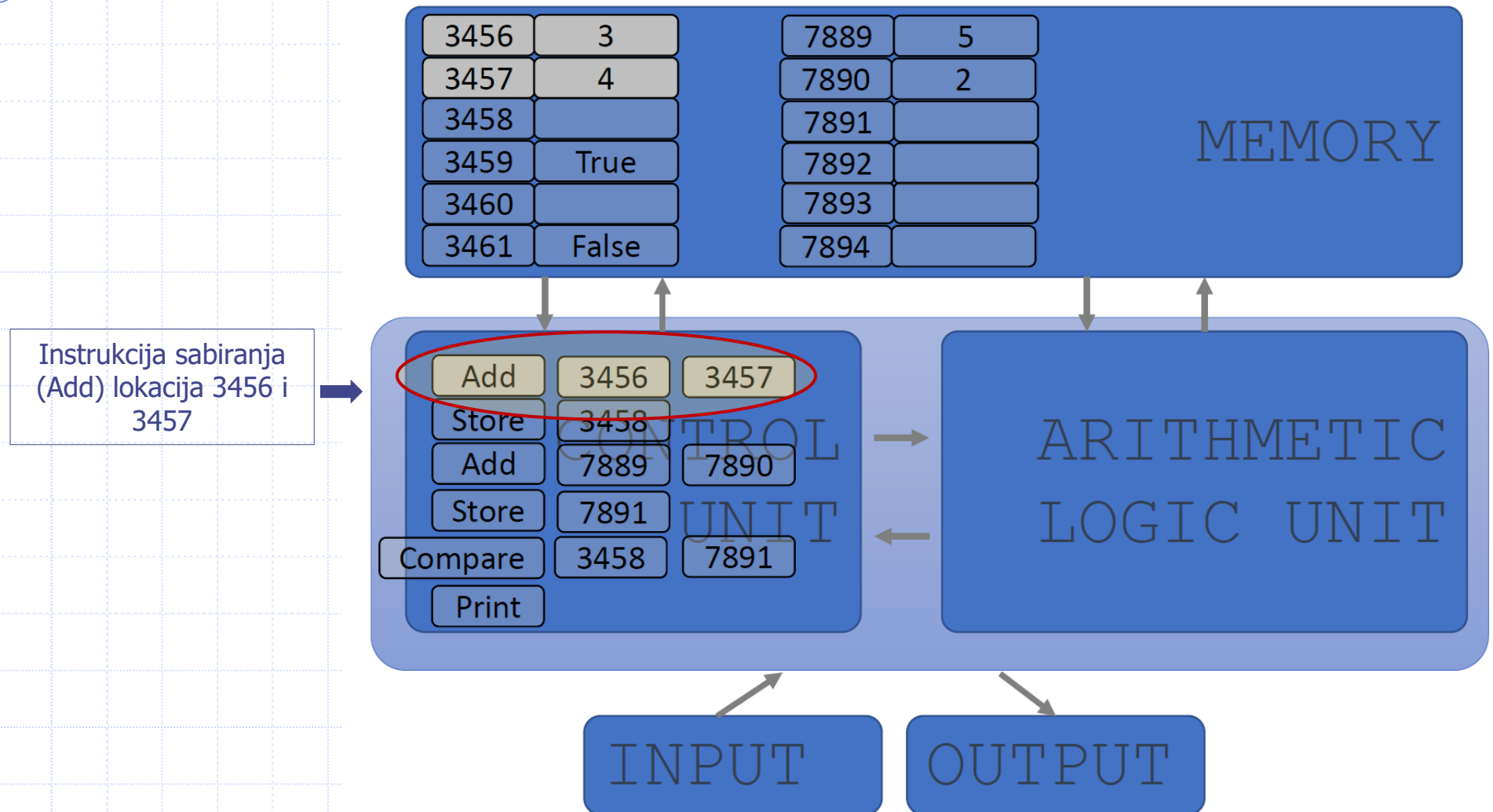
Arhitektura računara

Izvršavanje programa - primer



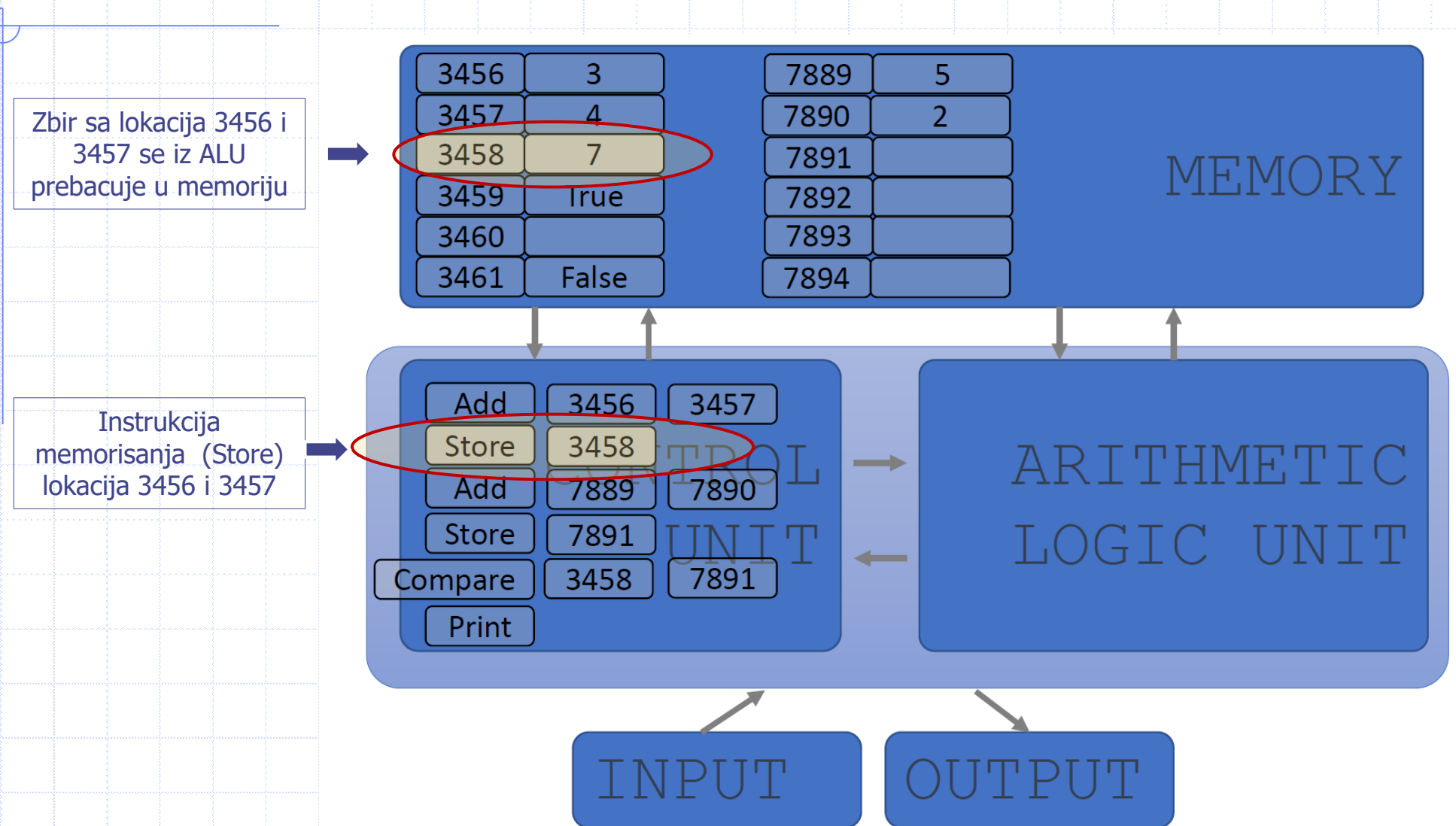
Na osnovu programskih instrukcija vrši se obrada podataka, izvršavanje tih instrukcija prema njihovom redosledu vrši poseban program INTERPRETER

Izvršavanje programa - primer



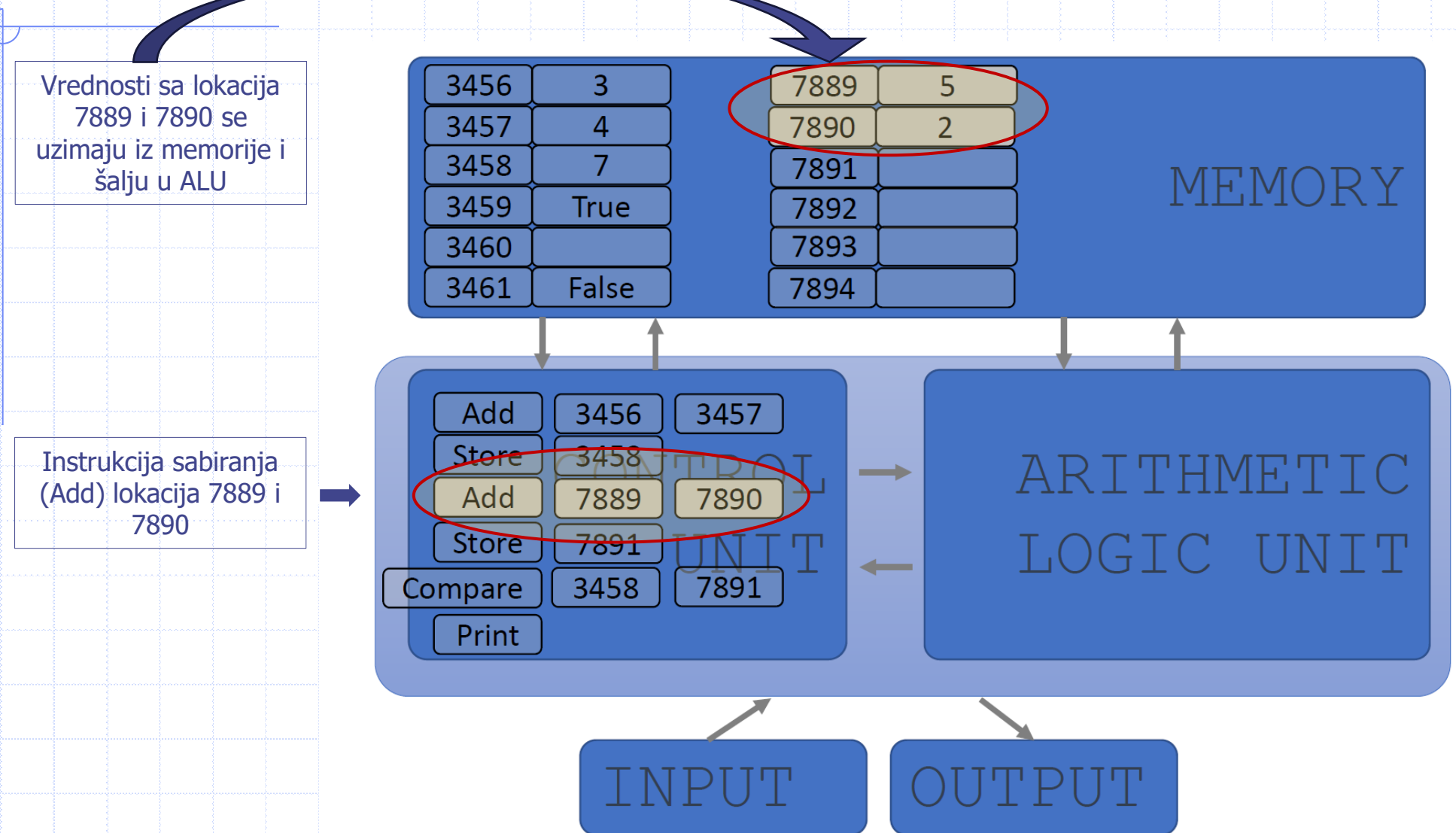
Program sa instrukcijama se nalazi u kontrolnoj jedinici, i kada se pokrene prvi put interpreter „vidi“ prvu instrukciju (Add-sabiranje) 2 vrednosti na memorijskim lokacijama 3456 i 3457, uzimaju se vrednosti sa te dve lokacije i šalju u ALU gde se vrši sabiranje

Izvršavanje programa - primer



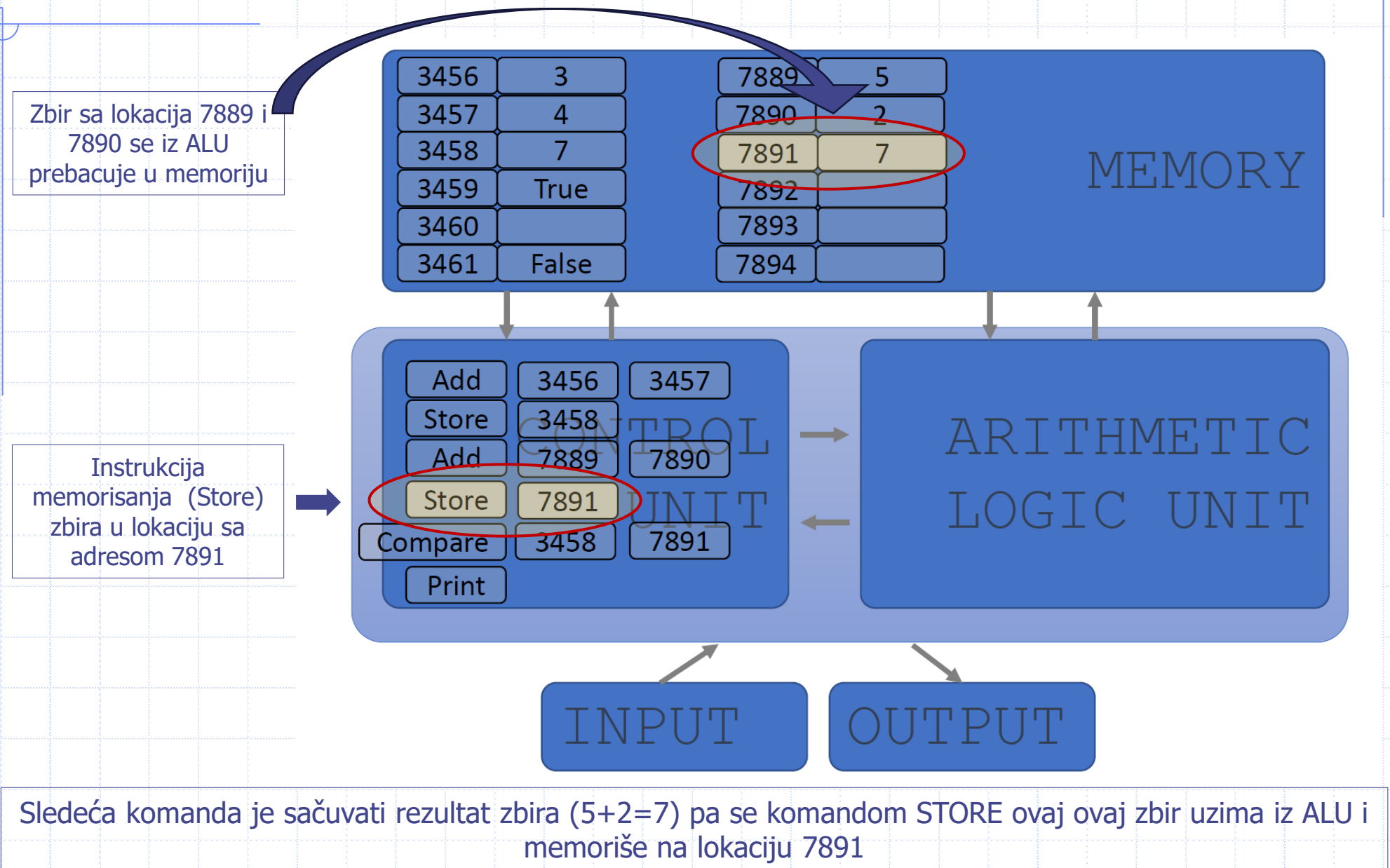
Sledeći korak je negde sačuvati rezultat zbira ($3+4=7$) pa se komandom STORE ovaj ovaj zbir uzima iz ALU i memoriše na lokaciji 3458

Izvršavanje programa - primer

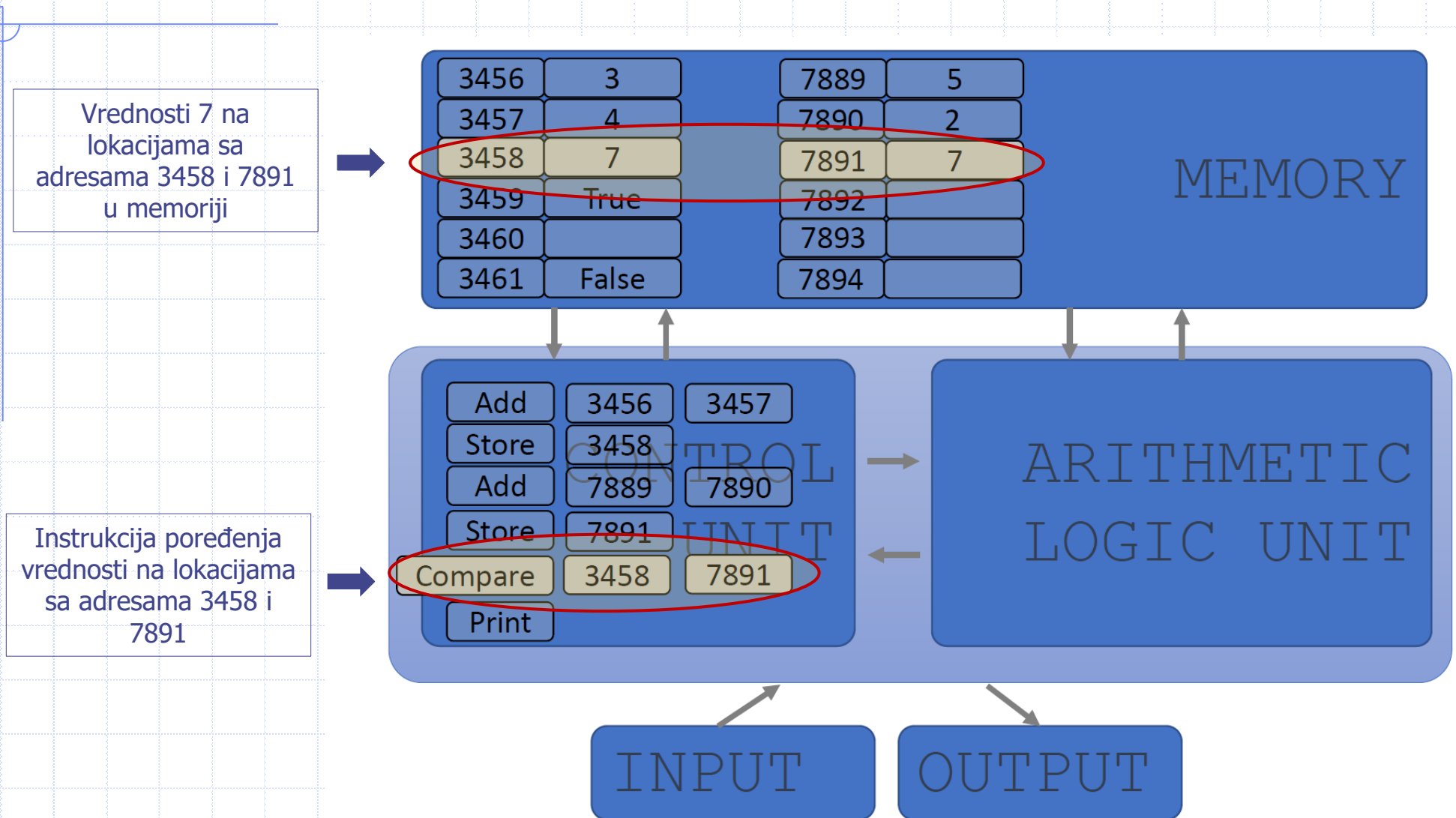


Sledeća komanda je sabiranja vrednosti sa memorijskih adresa 7889 i 7890, vrednosti se uzimaju iz memorije i šalju u ALU gde se vrši njihovo sabiranje

Izvršavanje programa - primer

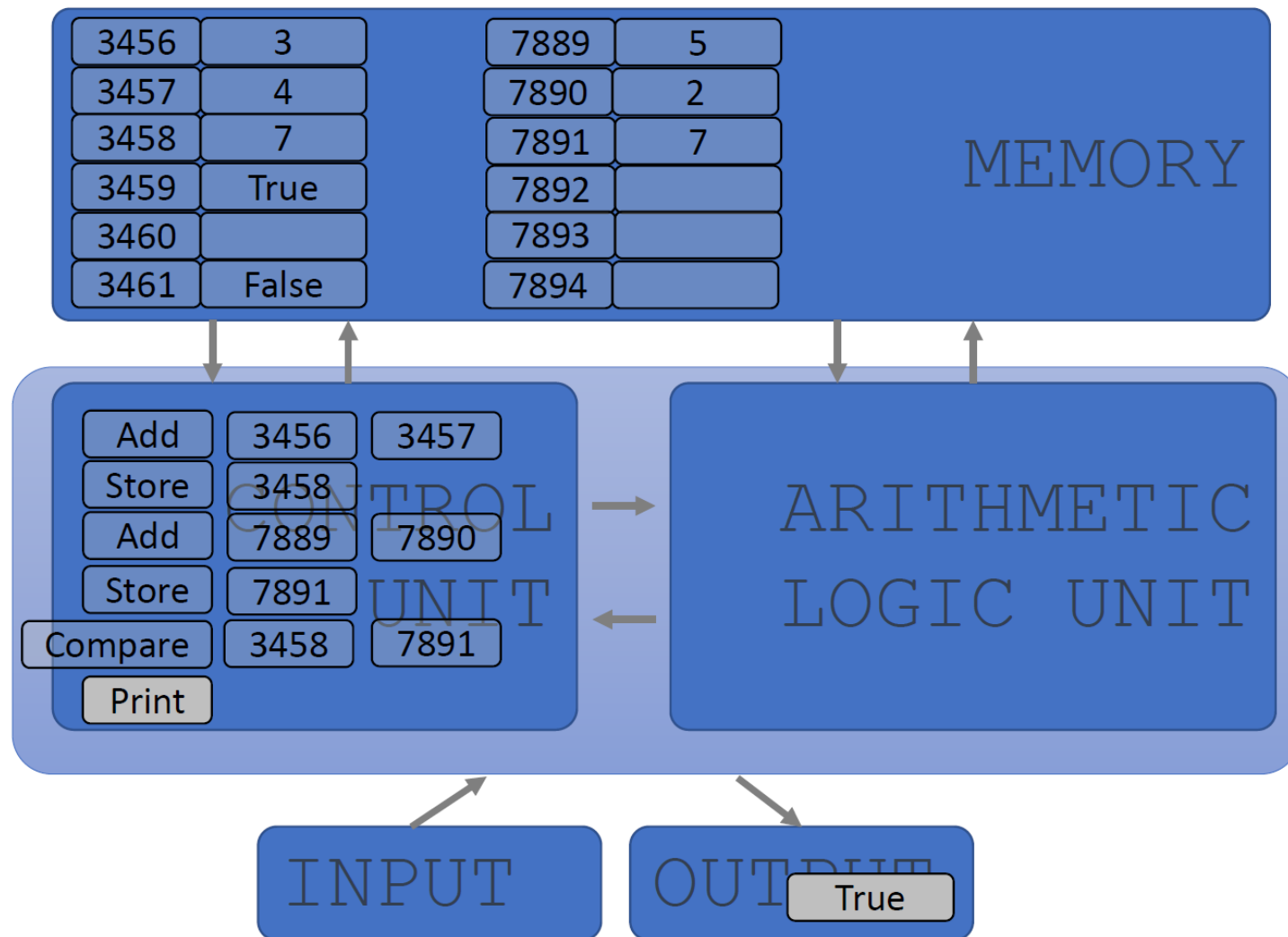


Izvršavanje programa - primer



Sledeća komanda je poređenje rezultata dva zbira zbira, pa se komandom COMPARE porede vrednosti sa lokacija 3458 i 7891, ALU vrši njihovo poređenje i vraća vrednost True interpreteru

Izvršavanje programa - primer



Poslednja komanda je komanda Print za prikaz rezultata poređenja vrednosti na lokacijama 3458 i 7891, kao izlaz se prikazuje True jer su vrednosti jednake

ASPEKTI PROGRAMSKIH JEZIKA

■ Primitivni konstruktori

- Engleski jezik: reči
- Programski jezik: brojevi, stringovi, aritmetički operatori

■ Sintaksa

- Engleski: "cat dog boy" → sintaksno neispravno
"cat hugs boy" → sintaksno ispravno
- Programski jezik: "hi"5 → sintaksno neispravno
"hi"*5 → sintaksno ispravno

ASPEKTI PROGRAMSKIH JEZIKA

- **Statička semantika**: da li sintaksno ispravan string ima značenje
 - Engleski: "I are hungry" → sintaksno ispravno
ali je statička semantička greška
 - Programski jezik: "hi"+5 → sintaksno ispravn
ali je statička semantička greška
- **Semantika**: značenje sintaksno ispravnog stringa koji nema semantičku grešku
 - Engleski: može imati više značenja "The chicken is ready to eat."
 - Programi imaju SAMO JEDNO značenje
- **Ali to značenje ne mora biti ono što programer ima nameru da uradi**

GDE NASTAJU GREŠKE?

- **Sintaksne greške**

- Najčešće i lako se otkrivaju

- **Statičke semantičke greške**

- Pojedini programski jezici ove greške proveravaju pre pokretanja programa
- Mogu uzrokovati nepredvidivo ponašanje programa

- Nema lingvističkih grešaka, ali je **različito značenje od onog šta programer očekuje**

- Program se prekida, zaustavljanje izvršavanja
- Program se izvršava beskonačno
- Program daje pogrešne odgovore!

OBJEKTI

- Programi manipulišu **objektima podataka (data objects)**
- Objekti imaju **tip** na osnovu kojeg program zna kako će te objekte procesirati
 - 30
 - Je broj
 - Može se dodati add/sub/mult/div/exp/etc
 - 'Ana'
 - Je sekvenca karaktera (string)
 - Može se dobiti substring, ali se ne može podeliti brojem

OBJEKTI

- **Skalarni**
 - Brojevi: 8.3, 2
 - Logičke vrednosti: True, False
- **Non-scalar** (imaju internu formu ili strukturu kojoj se može pristupiti)
 - Liste
 - Rečnici
 - Sekvence karaktera: "abc"

SKALARNI OBJEKTI

- `int` – celobrojni **integers**, npr. `5`, `-100`
- `float` – realni brojevi **real numbers**, npr. `3.27`, `2.0`
- `bool` – logičke **Boolean** vrednosti `True` i `False`
- `NoneType` – **specijalni** i nemaju vrednost, `None`
- Može se koristiti `type()` da se vidi tip objekta

```
>>> type(5)
```

```
int
```

```
>>> type(3.0)
```

```
float
```

int

0, 1, 2, ...
300, 301 ...
-1, -2, -3, ...
-400, -401, ...

float

0.0, ..., 0.21, ...
1.0, ..., 3.14, ...
-1.22, ..., -500.0 , ...

bool

True
False

NoneType

None

KONVERZIJE TIPA (CASTING)

- Može se **konvertovati objekat jednog tipa u drugi**
 - `float(3)` konvertuje `int 3` u `float 3.0`
 - `int(3.9)` konvertuje `float 3.9` u `int 3`
- Neke operacije vrše **implicitnu konverziju**
 - `round(3.9)` vraća `int 4`

IZRAZI

- **Kombinovanje objekata i operatora** za formiranje izraza
 - 3+2
 - 5/3
- Izraz ima **vrednost**, koja je određenog tipa
 - 3+2 ima vrednost 5 i tipa je **int**
 - 5/3 ima vrednost 1.666667 i tipa je **float**
- Python izračunava izraze i čuva njihove vrednosti. **Nikada se ne čuva sam izraz!!!**
- Sintaksa za jednostavan izraz
<objekat> <operator> <objekat>

PRIMERI

- `>>> 3+2`

- `5`

- `>>> (4+2) * 6 - 1`

- `35`

- `>>> type((4+2) * 6 - 1)`

- `int`

- `>>> float((4+2) * 6 - 1)`

- `35.0`

Do computations left to right – like in math!

Do computations inside parens first, left to right

Take care about what operations you are doing

OPERATORI za `int` i `float`

- $i + j$ → **zbir**
 - $i - j$ → **razlika**
 - $i * j$ → **množenje**
 - i / j → **deljenje**
 - $i // j$ → **celobrojno deljenje**
 - $i \% j$ → **ostatak deljenja** i sa j
 - $i ** j$ → i na **stepen** j
- Ako su oba `int`, rezultat je `int`
Ako je jedan ili oba tipa `float`, rezultat je `float`
Rezultat je uvek `float`

MNOGO OBJEKATA, šta se sve
može uraditi sa njima?!

a = 2 temp = 100.4
b = -0.3 go = True
x = 123 flag = False
small = 0.001 n = 17

PROMENLJIVE

- Računarske promenljive se **razlikuju** od matematičkih promenljivih

- **Math promenljive**

- Apstraktne
 - Mogu predstavljati više vrednosti

$$a + 2 = b - 1$$

$$x * x = y$$

- **Računarske promenljive**

- Imaju **samo jednu vrednost u datom trenutku**
 - Vezuje se za izraz
(izraz izračunava jednu vrednost!)

$$a = b + 1$$

$$m = 10$$
$$F = m * 9.98$$

one variable

one value

POVEZIVANJE PROMENLJIVIH sa VREDNOSTIMA

- U računarstvu, znak = je **dodela**
 - Jedne vrednosti jednom imenu promenljive
 - Znak = **nije jednakost**
- Dodela znači veza vrednosti sa imenom

variable pi = 355/113 value

- **Korak 1:** Izračunati vrednost na **desnoj strani** (VALUE)
 - Vrednost je sačuvana u memoriji
- **Korak 2:** Povezati **levom stranom** (VARIABLE)
 - Dobijanje vrednosti pridružene imenu pozivom tog imena

APSTRAKTNI IZRAZI

- Zašto se **daju imena** vrednostima izraza?
 - Da bi se **ponovo koristila imena** umesto vrednosti
 - Kod se lakše menja i čitljiviji je
- Birati imena promenljivih „mudro“
 - Kod treba da je čitljiv
 - Danas, sutra, sledeće godine
 - Čitamo ga ili mi ili drugi programeri
 - Slova, `_`, ne počinjati brojem....

```
#Compute approximate value for pi
```

```
pi = 355/113
```

```
radius = 2.2
```

```
area = pi * (radius**2)
```

```
circumference = pi * (radius*2)
```

comments start with a # and
are not part of code executed
– used to tell others what your
code is doing

an assignment
* expression on right
* variable name on left

NAJBOLJI STIL KODIRANJA?

```
#do calculations  
a = 355/113 * (2.2**2)  
c = 355/113 * (2.2*2)
```

meh

```
p = 355/113  
r = 2.2  
#multiply p with r squared  
a = p*(r**2)  
#multiply p with r times 2  
c = p*(r*2)
```

ok

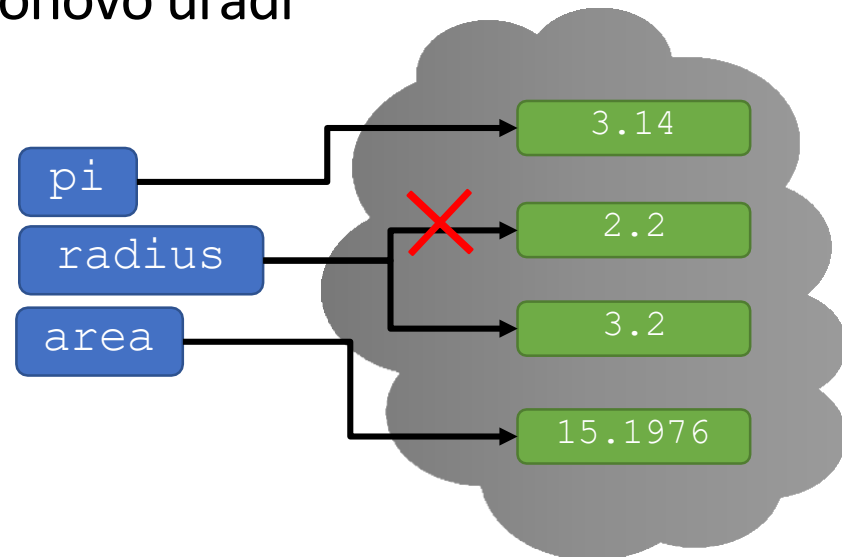
```
#calculate area and circumference of a circle  
#using an approximation for pi  
pi = 355/113  
radius = 2.2  
area = pi*(radius**2)  
circumference = pi*(radius*2)
```

best

PROMENA VREDNOSTI PROMENLJIVE

- **re-bind promenljive** se vrši dodelom nove vrednosti
- Prethodna vrednost je možda i dalje sačuvana u memoriji, ali se gubi njena „oznaka“
- Primer: Vrednost **površine kruga se ne menja** sve dok se ne kaže računaru da ponovo uradi kalkulaciju

```
pi = 3.14  
radius = 2.2  
area = pi*(radius**2)  
radius = radius+1
```



PRIMER!

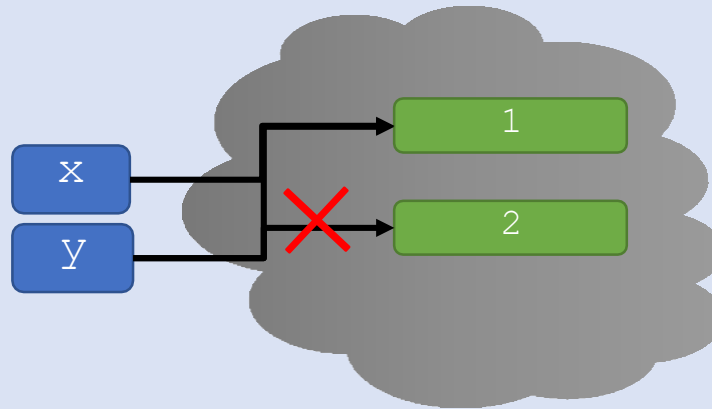
- Zameniti vrednosti za **x** i **y** bez direktnog povezivanja sa njihovim vrednostima

$x = 1$

$y = 2$

$y = x$

$x = y$



ODGOVOR:

